

How can a robot place an item on a cluttered desk

Ahmad Adnan Kabir

adnan.kabir@sabanciuniv.edu

Mechatronic Engineering, 2nd Year

Melih Kaan Şahinbaş

melih.sahinbas@sabanciuniv.edu

Computer Science and Engineering, 2nd Year

Aksel Dindisyan

aksel.dindisyan@sabanciuniv.edu

Computer Science and Engineering, 2nd Year

Hüseyin Doğan Türk

dogan.turk@sabanciuniv.edu

Computer Science and Engineering, 2nd Year

Yunus Emre Danabaş

yunusdanabas@sabanciuniv.edu

Mechatronic Engineering / Electronics Engineering (DM), 3rd Year

Steven El Khaldi

stevenelkhaldi@sabanciuniv.edu

Computer Science and Engineering, 3rd Year

Supervisors

Volkan Patoglu

Mechatronics Engineering

Esra Erdem

Computer Science and Engineering

Abstract

Let's consider that we have a service robot, a table, and a set of items. We want to place all the items one by one to the table by using a service robot. What kind of an approach can we use in order to place all the items to the table? Which algorithms give us the best result? That is the question that we studied, and we found various approaches such as using a grid, approximating items to rectangles and using greedy algorithms. We also looked at how to implement this algorithm. We examined the Baxter robot and its pick and place capabilities.

Keywords:- Pick and Place, Rectangle, Packing Problem, Grid

1 Introduction

Our research question is “How can a robot place an item on a cluttered desk?”. In order to solve this problem first, we needed to define our table and objects in a more structured way. First of all, we worked on a 2D system. We defined table as 2D coordinate system and objects as 2D objects. Then we examined similar researches. We found that in the game industry using bounding boxes and bonding circles are very common practices for collision detection (Mileff, 2023). Because using bounding boxes makes calculations very easier in order to detect collisions. One of the algorithms we used was dividing table to grids and then placing items accordingly. By doing that we prevented items from touching each other. But this method has some flaws that it produced extra blank spaces that stayed unused. Another algorithm we used was greedy packing algorithm. Since we can use bounding rectangle bounding boxes, we can convert this problem to a packing problem and after that we can solve it by a packing algorithm. Since rectangle packing is a NP complete problem solving it with a Brute force algorithm would be very inefficient and time consuming. So we used a greedy packing algorithm. Our greedy algorithm places items one by one by starting from corners and chooses the local optimal solution by analyzing CCOAs(Candidate corner-occupying action). (Mao Chen, 2007).

2 Definitions

We have defined our table as a 2D xy coordinate system. You can see it in figure 1. We are considering our objects as rectangles because we are using rectangle bounding boxes . We have defined them as dictionaries which hold min point coordinate, max point coordinate, corner points coordinate list, half size of the width, half size of the height and a color in order to better represent them. You can see an example object in figure 2. The reason we keep the half size of the width and height instead of width and height is that by using half sizes we can make calculations faster. (Mileff, 2023)

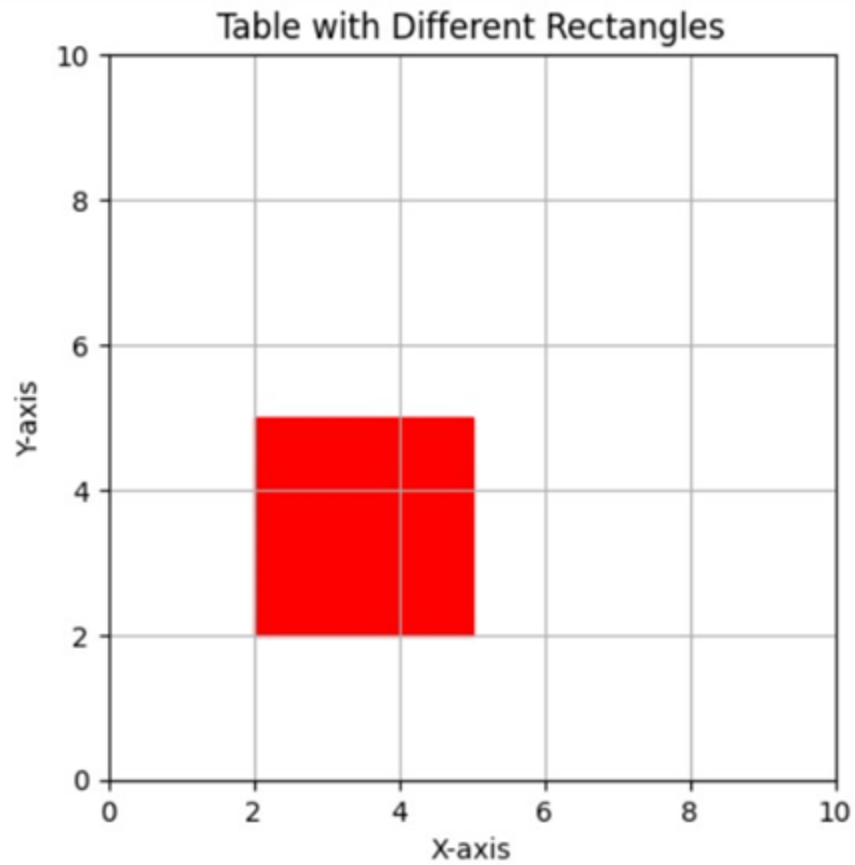


Figure 1

```
1 exampleRectangle = {'min_point':(2, 2),  
2                     'max_point': (5, 5),  
3                     'bb_points':[(2,2),(5,2),(2,5),(5,5)] ,  
4                     'half_size': (1.5, 1.5),  
5                     'color': 'r'}
```

Figure 2

3 Algorithm

3.1 Finding Blank Spaces

If there are already existing items on the table, then we need to first find out where are the blank spaces. For that I created a function called blank space creator. I give the list of objects, starting coordinate of the algorithm and the maximum coordinate that it can consider as inputs. After the algorithm finds the first blank space then it creates two recursive instances which starts from its other corner points which aren't minimum or maximum point. For example, you can see a randomly generated table in figure 3. Let's give the object list to our function. First it will start from (0,0) coordinate of the table. Then it finds the maximum coordinate it can go and takes the in between as a blank space and ads it to the blank space list. And then it starts two recursive functions from two separate corner of the blank space rectangle as you can see from the figure 4 where an intermediate step is represented. The black stars in the corner represents the new starting point of the two other recursive functions. And the gray rectangle is the representation of the newly formed blank space. In figure 5 you can see the end result where all the blank spaces are found. After finding all the blank spaces then we will place items to our blank spaces by using our greedy algorithm.

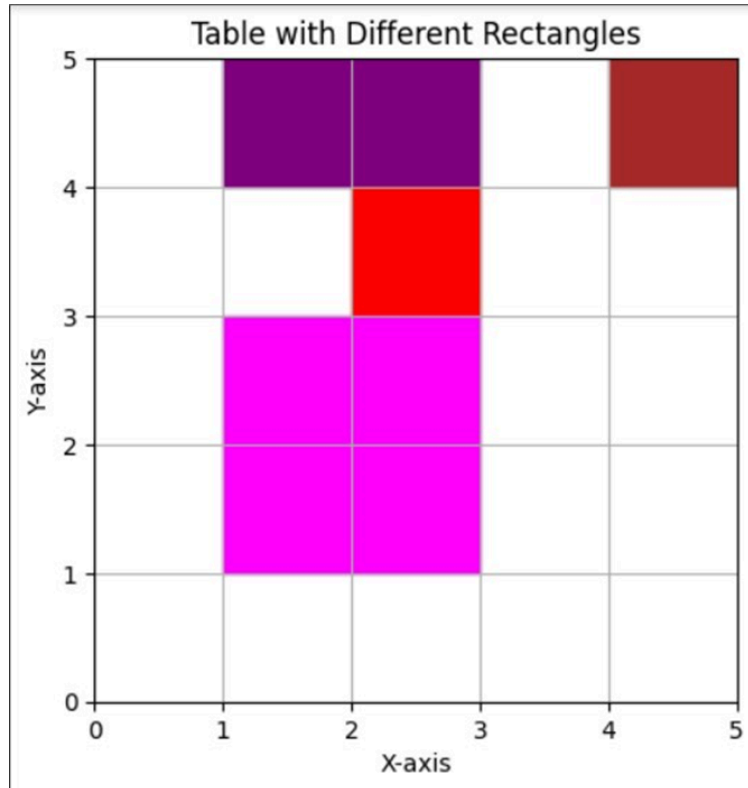


Figure 3

How can a robot place an item on a cluttered desk

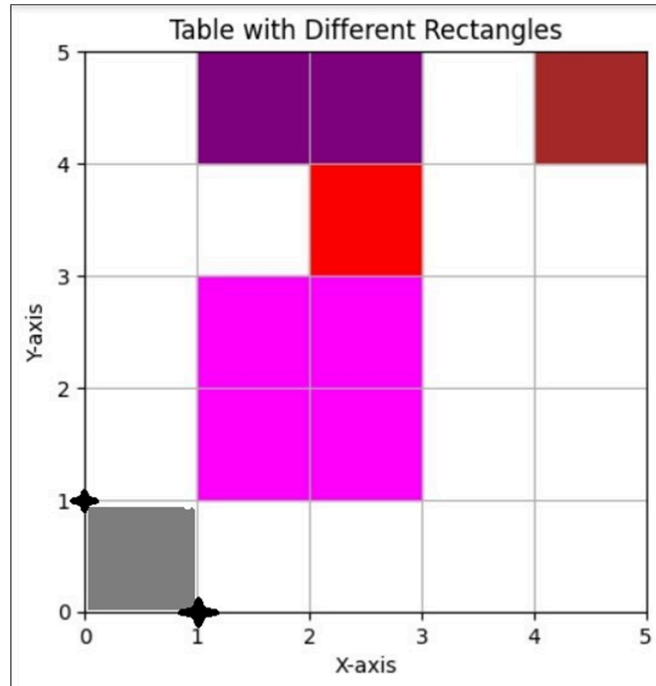


Figure 4

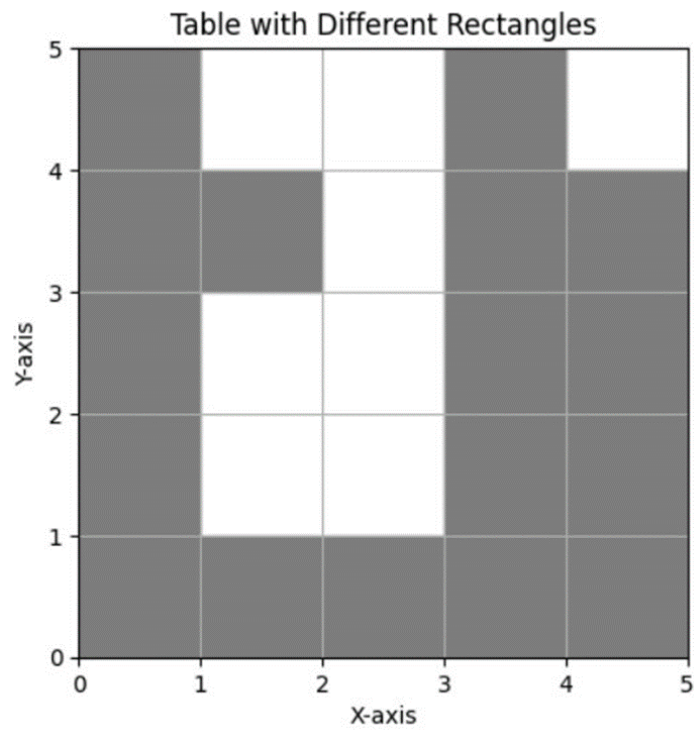


Figure 5

3.2 Greedy Algorithm

In our greedy algorithm we have a list of rectangles with different sizes and an empty coordinate system. Whenever we place a rectangle at least two sides of the rectangle should intersect with another rectangle or one of the sides of the table. At first we can only start from one of the corners of the table. Also before placing the item we need a table degree of CCOAs(Candidate corner occupying action). You can see the formula of the CCOA in the figure 6. And then we need to prioritize the action with the highest degree. Then we update the rectangle list and list of rectangles that are placed. We do this until either all of the items are placed or we are unable to place any additional items to the table. (Mao Chen, 2007). In figure 7, you can see the pseudocode. As I mentioned, first it calculates all possible CCOAs and then chooses the highest one. After that it modifies items remaining and items placed. As I mentioned this is a greedy algorithm so it finds the locally optimal solutions and hopes that it will lead it to the globally optimal solution. So it doesn't guarantee an optimal solution but it is much faster than its brute force counterpart. Its time complexity is $O(n^4)$. (Mao Chen, 2007).

$$\lambda = 1 - d_{\min} / r \left(\frac{w_i + l_i}{2} \right)$$

Figure 6 (Mao Chen, 2007).

Procedure $A_0(C, L)$

Begin

While(L is not empty)

For each CCOA in L

 Calculate the degree;

 Select the CCOA (i, x, y, θ) with the highest degree;

 Modify C by placing rectangle i at (x, y) with orientation θ ;

 Modify L according to the new configuration C ;

 Return C ;

End.

Figure 7 (Mao Chen, 2007).

3.3 AABB Tree Algorithm

Axis-Aligned Bounding Box (AABB) trees are a type of Bounding Volume Hierarchy (BVH) used extensively in computer graphics, collision detection, and physics simulations. A set of primitives, such as spheres, triangles, or other geometric objects, are arranged into a tree structure by them, with each node denoting an AABB that confines a subset of the primitives. This part describes the partitioning strategy, attributes, and performance aspects of the resulting AABB tree construction algorithm.

Recursive subdivision is used in a top-down fashion to form the AABB tree. The following are the algorithm's primary steps:

In order to create an AABB hierarchical volume bounding tree, we first determine which AABB bounds the collection of primitives we are working with. The longest axis of this AABB is then found. Next, a dividing plane orthogonal to this longest axis is selected, and it is placed at the median of the primitives' midpoints along this axis. Following that, the primitives are divided into two subsets according to where they are in relation to the partitioning plane. Each subset is subjected to a recursive procedure that involves computing the AABB, determining the longest axis, selecting a dividing plane, and separating the primitives until each subset has just one primitive, at where a leaf node is formed. This process of recurrent subdivision never ends until the full tree is built. When a subset includes a single primitive, the recursion ends, adding a leaf node to the tree.

Procedure BuildAABBTREE(primitives):

Begin

 If number of primitives == 1:

 Return new Node with AABB(computeAABB(primitives)) and the single primitive

 aabb = ComputeAABB(primitives) axis = Longest axis of aabb midpoints = List of midpoints of all primitives along axis median = Median value of midpoints

 left_primitives, right_primitives = PartitionPrimitives(primitives, axis, median)

 If left_primitives is empty or right_primitives is empty:

 Split primitives into two nearly equal subsets left_primitives = first half of primitives

 right_primitives = second half of primitives

 left_node = BuildAABBTREE(left_primitives)

 right_node = BuildAABBTREE(right_primitives)

 Return new Node with aabb, left_node, and right_node

End

The AABB hierarchical volume bounding tree is a powerful structure for spatial partitioning in robotics. Fast intersection testing, collision detection, and visibility determination are made possible by the tree's efficient hierarchy-building of primitives by recursive subdivision based on median partitioning along the longest axis. The presented algorithm strikes a compromise between performance and simplicity, making it appropriate for a variety of uses.

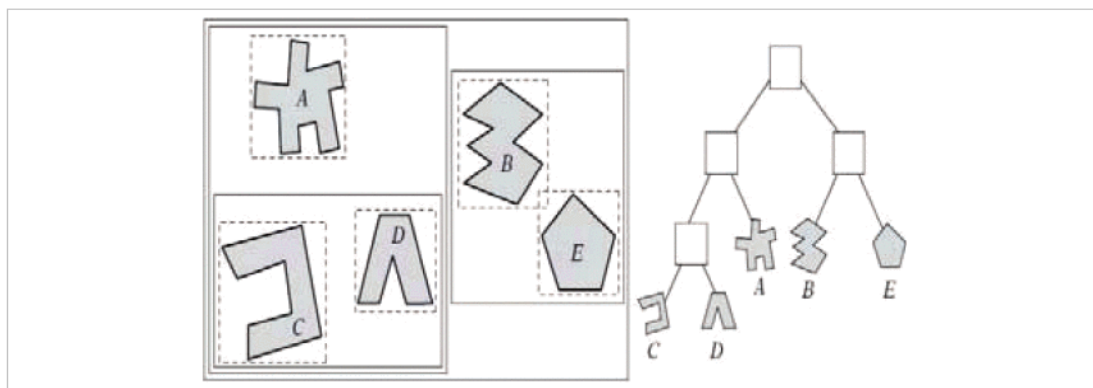


Figure 8.

We experimented with several partitioning coordinates (β), and found that the optimum result is obtained when β is set to the median of the AABB, which divides the box into two equal parts. When using this heuristic, the worst-case time to build an AABB tree for n primitives might be $O(n^2)$; however, in the typical scenario, when the primitives are distributed roughly evenly over the box, the time to build an AABB tree is only $O(n \log n)$ (Xing, Liu, & Xu, 2010).

3.4 Gridization-Based Hierarchical Placement Algorithm

Our novel approach to placing items on a cluttered desk employs a hierarchical gridization technique that iteratively subdivides the table space into finer grids, coupled with a graph-based spatial checking mechanism to ensure efficient placement and collision avoidance. The algorithm initiates by defining the table as a 2D coordinate system and dividing it into an initial grid. Each cell within this grid is evaluated for its occupancy status. If no empty cells are detected at this level, the grid resolution is incrementally increased by a factor of $\sqrt{2}$, and the search is repeated. This recursive refinement continues until an empty cell is identified or a predefined grid resolution limit is reached.

Upon identifying potential placement areas, the algorithm constructs a graph where nodes represent already placed items and edges denote adjacency, starting from the item with the smallest x-coordinate. This graph aids in evaluating the spatial feasibility of placing new items. The algorithm then checks for any grid cells around the vertices of this graph that are at least 70% empty. If an appropriate placement area is found, the item is positioned, followed by collision detection to ensure no overlap occurs. If a collision is detected, the item is slightly adjusted, and the process is repeated until a collision-free placement is achieved.

In cases where the graph-based check does not identify a suitable placement, the algorithm performs a random search of empty cells, checking for space and fit for the object and its neighbors. If no space is found through this method, the grid resolution is further increased, and the algorithm reinitiates the process. This hierarchical and iterative approach efficiently balances optimal placement with computational efficiency, making it suitable for real-time applications in service robotics. Future enhancements may include extending the algorithm to 3D environments and incorporating more sophisticated collision detection techniques. This methodology ensures a structured and systematic way to maximize space utilization while minimizing computational complexity.

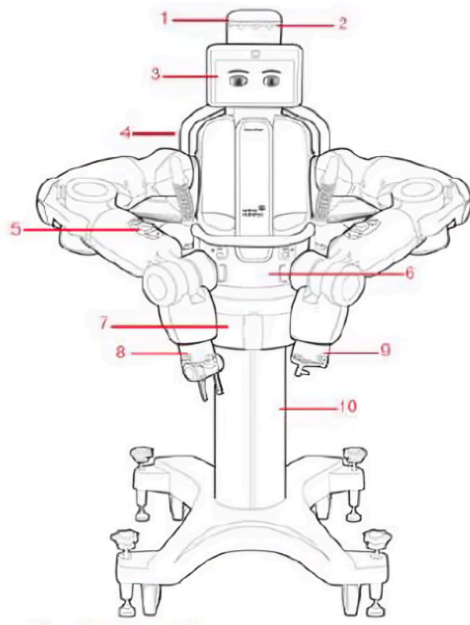
4 Implementation: Baxter Control Using ROS

This section of the report presents the development and implementation of a pick-and-place task using the Baxter robot, controlled through ROS and MoveIt!. The project was conducted in both simulated and real-world environments, utilizing Ubuntu 16.04 with ROS Kinetic for real-world implementations and Ubuntu 20.04 with ROS Noetic and Gazebo 11 for simulations.

4.1 Baxter Robot Features

The Baxter robot, developed by Rethink Robotics, is designed for collaborative tasks in industrial and research environments. Its primary features include:

- **Dual 7-DOF Arms:** Each arm has seven degrees of freedom, providing flexibility and precision in movement.
- **2-DOF Head with Animated Face:** The head can tilt and pan, and features an animated face for visual feedback and interaction.
- **Vision System:** Baxter is equipped with three cameras—one on the head and one on each arm—enabling it to perceive and interact with its environment.
- **Safety and Control Systems:** It includes advanced safety systems and collision detection sensors to ensure safe operation around humans.



1. Condition Ring
2. Attention Ring
3. Display
4. Navigator (torso)
5. Navigator (arms)
6. Lower Front Panel (access to air valves)
7. Waist (Zero level for Baxter)
8. Training Cuff with Electric Parallel Gripper
9. Training Cuff with Vacuum Cup Gripper
10. Pedestal (adjusts to two heights)

Baxter has a Camera in each End Effector and a Camera above the Display.

Figure 9: Schematic representation of the Baxter robot.

4.2 Overview of ROS

Robot Operating System (ROS) is an open-source, flexible framework for writing robot software. It provides a collection of tools, libraries, and conventions to simplify the task of creating complex and robust robot behavior across a wide variety of robotic platforms. ROS is essential for integrating various robotic components seamlessly, facilitating communication between different subsystems through message passing, and enabling modular and reusable code development.

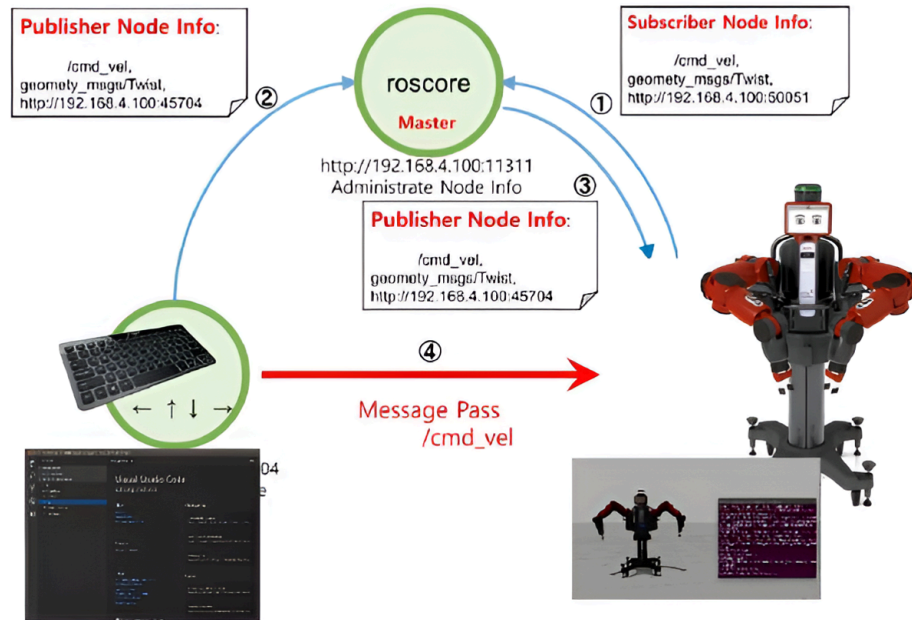


Figure 10: Message communication in Baxter robot with ground station (PC) using ROS.

4.3 MoveIt! Trajectory Planning

MoveIt! is a powerful ROS-based software for mobile manipulation, providing tools for developing advanced robotics applications. It includes capabilities for motion planning, manipulation, 3D perception, kinematics, control, and navigation. MoveIt! enables users to plan and execute motions for the robot's arms, ensuring precise and collision-free operations.

4.3.1 Using MoveIt!

In the context of this project, MoveIt! was utilized for trajectory planning, allowing the Baxter robot to determine and execute paths for pick-and-place tasks. The interface provides visual markers to define the end-effector's target position and orientation, making it intuitive to program complex movements.

How can a robot place an item on a cluttered desk

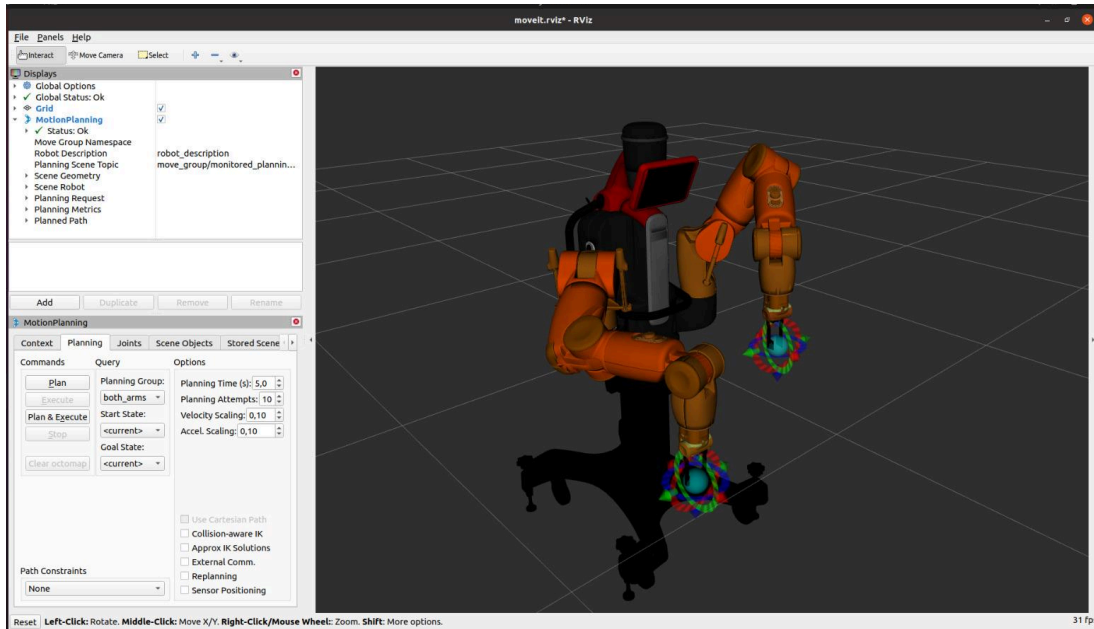


Figure 11: MoveIt! interface used for planning and visualizing Baxter's arm trajectories.

In the MoveIt! interface, two significant markers allow for adjusting the end effector's position and orientation. This visual feedback simplifies the process of defining the robot's movements and can be used for future implementations of various trajectory planning algorithms.

Additionally, you can watch the demonstration of manipulating Baxter in a real-world scenario using MoveIt! in the following video:

- [Manipulating Baxter in Real World Using MoveIt!](#)

4.4 Pick and Place Implementation

The primary objective of the pick-and-place project is to design and develop an intelligent method for a service robot to place a book on a cluttered desk. If sufficient space is available, the robot places the book directly; otherwise, it rearranges other items to create the necessary space.

4.4.1 Simple Pick and Place Demo

As an initial step, a simple pick-and-place demo was implemented where the Baxter robot picks up a paper cup from one location and places it in another. This was tested both in a simulated environment using Gazebo and on the actual Baxter robot. ROS and RViz provided the visualization tools necessary for observing the different views from Baxter's cameras during the operation.

To facilitate this task in ROS Noetic, it was necessary to resolve several compatibility issues, as there is no official package for Baxter simulation in ROS Noetic. After extensive debugging, the package was successfully adapted for Noetic by us.

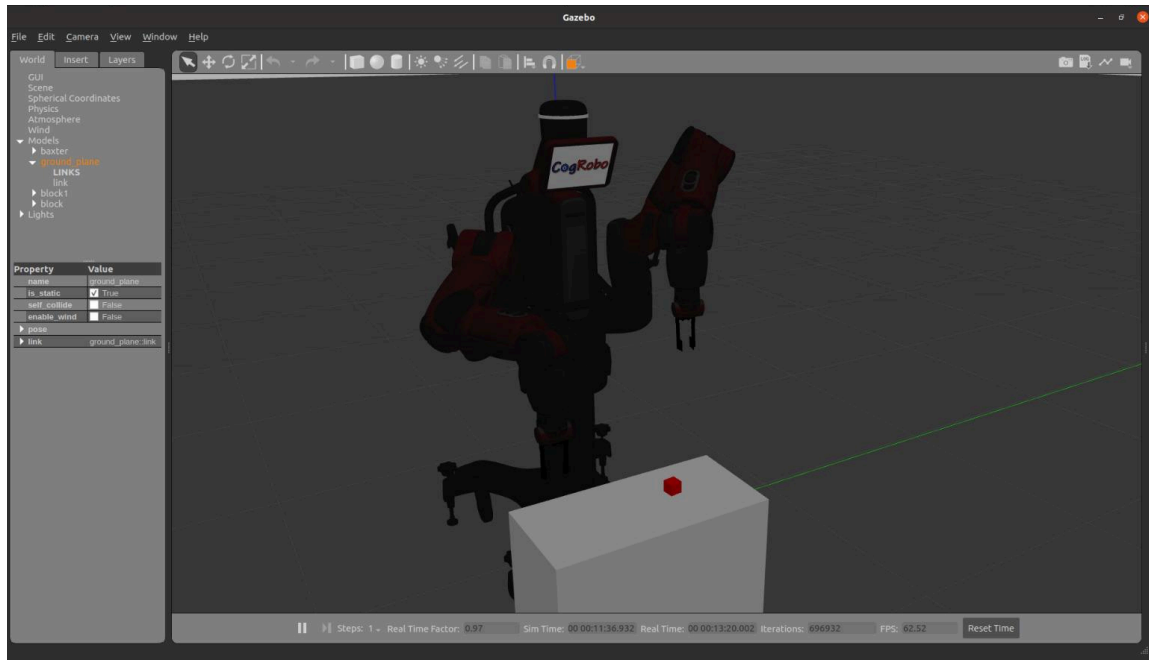


Figure 12: Baxter performing a pick-and-place task in Gazebo simulation.

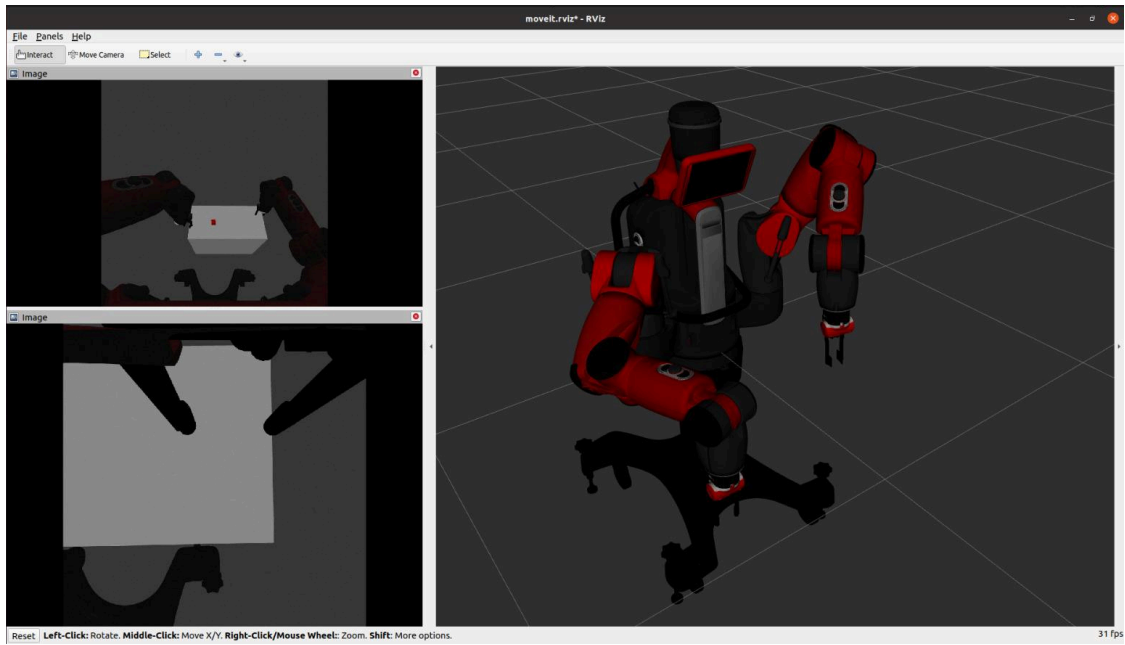


Figure 13: Baxter performing a pick-and-place task in Gazebo simulation (RViz Perspective).

How can a robot place an item on a cluttered desk



Figure 14: Baxter performing a pick-and-place task in a real-world scenario.

Additionally, you can watch the demonstrations of these tasks in the following videos:

- [Pick and Place in Gazebo Simulation and Manipulation Using MoveIt!](#)
- [Pick and Place in Real Life](#)

5 Discussion and Conclusion

This project demonstrates significant progress in controlling the Baxter robot using ROS and MoveIt!. The combination of these tools allows for sophisticated manipulation tasks, making Baxter an effective platform for service and industrial applications. The successful implementation of pick-and-place tasks, both in simulation and real life, showcases Baxter's capabilities and the robustness of the control systems developed.

By leveraging ROS for message communication, Baxter's built-in features like cameras and 7-DOF arms, and the advanced trajectory planning provided by MoveIt!, we have created a versatile and reliable system for robotic manipulation tasks. This work lays the foundation for more complex and intelligent robotic applications in the future.

In our algorithm development process, we initially redefined objects and the table into a compositional structure, enabling us to use them effectively in our algorithms. Subsequently, we developed an algorithm to identify the blank spaces between already placed items. Our research indicated that there are multiple ways to place an item on the table, each with its own advantages and disadvantages. Following this, we implemented three alternative algorithms to facilitate the placement of items on the table. Moving forward, we plan to optimize these algorithms and integrate them into our robotic system.

In the next semester, we plan to implement an advanced algorithm developed by the computer science students in our group. This algorithm will further enhance the Baxter robot's pick-and-place operations by incorporating more sophisticated decision-making processes.

Additionally, we need to deepen our understanding of the built-in inverse kinematic packages and the MoveIt! inverse kinematic package. Understanding how to effectively utilize these packages in our Python code will streamline the implementation process and allow for more precise control of the robot's movements.

References

- Mao Chen, W. H. (2007). *A two-level search algorithm for 2D rectangular*.
- Mileff, P. (2023). *COLLISION DETECTION IN 2D GAMES*.
- Y. -S. Xing, X. P. Liu and S. -P. Xu (2010). *Efficient collision detection based on AABB trees and sort algorithm*.

Appendix (Code for Pick and Place)

```
#!/usr/bin/env python
# importing necessary dependencies
import rospy
import baxter_interface
from baxter_interface import CHECK_VERSION

# Initialize ROS node
rospy.init_node("Pick_arm")
print("Getting robot state...")

# Enable the robot
robot_state = baxter_interface.RobotEnable(CHECK_VERSION)
init_state = robot_state.state().enabled
print("Enabling robot before starting the action...")
robot_state.enable()

# Initialize interfaces
head = baxter_interface.Head()
right_limb = baxter_interface.Limb("right")
right_gripper = baxter_interface.Gripper('right')

left_limb = baxter_interface.Limb("left")
left_gripper = baxter_interface.Gripper('left')

# Define positions
pick_positions = {}
pick_positions_up = {}
place_positions = {}
place_positions_up = {}

# Populate positions (example values)
pick_positions[1] = {
    'right_s0': 0.10431069357620813, 'right_s1': 0.6059224112147384, 'right_w0': -2.196276993054941,
    'right_w1': 1.291228328202547, 'right_w2': 0.1169660350762628, 'right_e0': 1.7867041226895357, 'right_e1':
0.8256651590793239
}
pick_positions_up[1] = {
    'right_s0': 0.0011504855909140602, 'right_s1': 0.46901462589596526, 'right_w0': -2.1445051414638083,
    'right_w1': 1.647111870991963, 'right_w2': 0.2998932440315984, 'right_e0': 1.896767244220314, 'right_e1':
1.1681263699747426
}
place_positions[1] = {
    'right_s0': 0.6258641614572488, 'right_s1': 0.5445631796993219, 'right_w0': -2.081228433963535,
```

```
'right_w1': 1.2252671543234743, 'right_w2': 0.6991117440787773, 'right_e0': 1.6459613854010489, 'right_e1':  
0.870150601928001  
}
```

```
place_positions_up[1] = {  
    'right_s0': 0.5250049246537829, 'right_s1': 0.2577087723647495, 'right_w0': -1.8438449070382674,  
    'right_w1': 1.4894953450367368, 'right_w2': 0.9085001216251363, 'right_e0': 1.6168157504312262, 'right_e1':  
1.1688933603686853  
}
```

```
pick_positions[2] = {  
    'right_s0': 0.08, 'right_s1': -1.0, 'right_w0': 1.19,  
    'right_w1': 1.94, 'right_w2': -0.67, 'right_e0': 1.03, 'right_e1': 0.50  
}
```

```
# Ensure gripper is open  
right_gripper.open()  
rospy.sleep(0.5)
```

```
# Move head to examine the environment  
head.set_pan(-1, speed=0.20, timeout=10)  
head.set_pan(1, speed=0.20, timeout=10)  
head.set_pan(0, speed=0.20, timeout=10)
```

```
# Perform the pick and place operation once  
print("Performing the cube moving action once. Ctrl+C quits the program.")  
i = 1 # We are doing this for the first set of positions
```

```
# Move head towards the object  
head.set_pan(-0.25, speed=0.20, timeout=10)
```

```
# Move to pick position above the object  
right_limb.move_to_joint_positions(pick_positions_up[i])  
rospy.sleep(0.5)
```

```
# Move down to the object  
right_limb.move_to_joint_positions(pick_positions[i])  
rospy.sleep(0.5)
```

```
# Close gripper to pick the object  
right_gripper.close()  
rospy.sleep(0.5)  
print("Gripper closed")
```

```
# Move back up with the object  
right_limb.move_to_joint_positions(pick_positions_up[i])  
rospy.sleep(0.5)
```

```
# Move head towards the place location
```

How can a robot place an item on a cluttered desk

```
head.set_pan(0.25, speed=0.20, timeout=10)

# Move to place position above the destination
right_limb.move_to_joint_positions(place_positions_up[i])
rospy.sleep(0.5)

# Move down to place the object
right_limb.move_to_joint_positions(place_positions[i])
rospy.sleep(0.5)

# Open gripper to release the object
right_gripper.open()
rospy.sleep(0.5)
print("Gripper opened and cube moved successfully.")

# Move back up after placing the object
right_limb.move_to_joint_positions(place_positions_up[i])
rospy.sleep(0.5)

# Move to pick position above the object
right_limb.move_to_joint_positions(pick_positions_up[1])
rospy.sleep(0.5)
```